that contains a part of the data of a partitioned table (horizontal slice). (source)| Partition
| |
| Track | A sub-group within the WG that tackles one scaling pattern. | | |

Overview

Our current architecture relies, almost exclusively and by design, on a single database to be the sole and absolute manager of data in terms of storage, consistency, and query results collation. Strictly speaking, we use a single logical database that is implemented across several physical replicas to handle load demands on the database backend (with the single pseudo-exception of storing diffs on object storage). Regular analysis of database load, however, is showing that this approach is unsustainable, as the primary RW database server is experiencing high peaks that are approaching the limits of vertical scalability.

We explored sharding last year and scoped it to the database layer. We concluded that while there are solutions available in the market, they did not fit our requirements, both in financial and product fit terms, as they would have forced us into a solution that was difficult (if not impossible) to ship as part of the product.

We are now kicking off a new iteration on this problem, where the scope is expanded from the database layer into the application itself, as we recognize this problem cannot be solved to meet our needs and requirements if we limit ourselves to the database: we must consider careful changes in the application to make it a reality.

Data management as a discipline

Deferring most, if not all, data management responsibities to the database has enabled us to offload decisions and rely on PostgreSQL's excellent capabilities to cope with application demands. While we have run into a few scalability limits (which were addressed through infradev), the database has, for the most part, held its ground, aided by copious amounts of hardware. This has provided specific fixes to very specific problems, and has afforded us development speed but blindsided us to long-term issues as the database grows. Spot fixes will not be sufficient to sustain growth on GitLab.com. This is now reflected in significant technical debt issues such as cibuilds, where the current schema and usage is making our ability to scale it difficult. Thus, we will need to take a more strategic view to provide a long-term, comprehensive solution, and adopt data management as a full-fledged discipline.

Data management as a discipline entains looking beyond a given schema definition to meet immediate application needs. We must achieve a long-term understanding of how the scheme will behave at scale (both in terms of performance and usability), whether we are asking too much of a given schema definition and refactoring is in order, growth and limitations, appropriate observabilily against each and every table, queries running against them, and predictions on their scalability limitations (for instance, PK growth).

There will be areas of the database where discipline and scalability will overlap. In general, we should tackle discipline first and scalability later.

Trade-offs

As we look at scaling the database backend, two primary trade-offs will dominate our decisions:

- Latency: Currently, we rely on synchonicity and available capacity to achieve acceptable levels of latency, which mostly happens . We will have to pay detailed attention to latency expectations and manage them accordingly.
- Complexity: scaling the database backend will inherently introduce additional complexity, which we must manage carefully.

Additionally, we will need to evolve and mature our data migration strategies. Over time, it will be difficult to perform migrations, even if these are carried in the background. Some of them will need to become part of the application as live, in-app migrations (for example, migrating a table into a new schema may need to happen opportunistically: when a record is read, a lookup takes place in the new table; failing that, the lookup happens in the old table and the result is both returned to the user and migrated into a new table).

Functional decomposition split

We are at the stage of having to embrace functional decomposition by separating data by meaning, function or usage, but we must do so in ways that enable the application to continue to view the database backends as a single entity. This entails the minimization of introducing new database engines into the stack while taking on some of the data management responsibilities.

Data access layer

We recognize the advantages of a single application and wish to maintain the view of a single data store as far down the stack as possible. It is imperative we maintain a high-degree of flexibility and low-resistance for developers, which translates into a cohesive experience for administrators and users. To that end, we will need to build and introduce a data access layer that the rest of the application can leverage to access backend data stores (which is not a new concept for GitLab, as the success of Gitaly clearly shows).

As we look at scaling the database backend, a global reliance on the database to be the sole executor of data management capabilities is no longer possible. In essence, the scope of responsibility is now more localized, and the application must accept some of these responsibilities. These cannot be implemented in an ad-hoc fashion, and should be centralized in a data access layer that, on the one hand, understands the various backends supporting data storage, and on the front-end, can service complex queries and provide the necessary caching capabilities to address the latencies that will be introduced by trying to scale the database horizontally, all while hiding the backend details from the rest of the application.

As the database backend is decomposed through the use of the tentatively proposed scaling patterns, the need to implement a data access layer becomes apparent. We can no longer rely on the database to perform all data composition duties. The application itself (or a service on its behalf) must take on those responsibilities.

Scaling Patterns

We must ensure we approach database scalability both systematically and holistically. We should not enable disjoined approaches to scaling the database. We therefore propose deciding on a

small number of patterns to break down the problem and provide solutions that can apply to a variety of tables, keeping the solution space small, manageable, and with a certain degree of flexibility. Once we understand the solutions applicable to these patterns, we can think holistically about the data access layer, enabling us to apply iteration to its development while minimizing disruption to the rest of the application.

Read-Mostly Data

There are small pockets of data that are frequently read but seldomly written, and rarely participate in complex queries (such as the license table). They should be offloaded to more appropriate backends that are better suited for high-frequency reads, minimally for caching purposes (since the only backend we consider to be durable is the database). While the queries are simple and the amount of data is small, these queries do contribute a significant amount of context switches in the database.

For details, see the blueprint.

Time-Decay Data

Some datasets are subject to strong time-decay effects, in which recent data is accessed far more frequently, or is more valuable, than older data. This effect is usually tied to product and/or application semantics, and we can potentially take advantage of this pattern.

We should consider some of the following patterns:

1. Partitioning based on time
1. Deleting old data which is no longer valuable
1. Shifting data to object storage

These patterns can also be used in combination. For example, partition based on time and drop partitions older than X months.

For details, see the blueprint.

Entity/Service

A popular database scalability methodology entails decomposing concrete entities (e.g., users) into separate data stores. This approach is typically associated with microservices, and while at times it may be useful to encapsulate an entity behind a service, this is not a strict requirement, especially within the context of our application (even though, as mentioned earlier, we have already done this with Gitaly).

This approach shifts consistency concerns to the application, as the database runtime cannot possibly track external dependencies. This strategy works particularly well when a portion of the column data can be streamlined and kept on the "main" database while offloading the rest of the metadata to a separate backend.

Sharding

One of the most widely used methodolgies to database scalability is sharding). It is also one

of the hardest to implement, as sharding shifts logical data consistency and query resolution demands into the application, significantly elevating the relevance of the CAP theorem in our design choices. CAP effects today are negligible: there is a replication delay, but so small it can be, and is, ignored. While these are not trivial problems to solve, they are solvable.

Asynchronicity and CAP

As we dive into the relevance of the CAP theorem, we should also consider it outside the context of sharding. This is a point @andrewn crystalized in a recent conversation where we discussed the topic of database vertically scalability. In this context, we should evaluate the need for full transactional synchronicity, and shift, where possible, to asynchronous mode.

The importance of nomenclature

It is important we settle on specific nomenclature. Currently, there is a fair amount of discussion on sharding, and while it is a part of this equation, it isn't the whole equation. Thus, let's make sure we talk about database scalability, y-axis split, etc appropriately.

Plan

1. Kick-off working group: handbook, agenda, meeting
1. Determine authoritative list of scaling patterns and determine a path for sharding to comprise the first iteration
    1. A blueprint per scaling pattern should be produced by an assigned DRI to:
        1. Describe the scaling pattern
        1. Applicable use cases in the database
        1. Analysis and evaluation of current use cases, their effects on the database
        1. A brief overview of the design
    1. A blueprint for the path of how to shard the database
        1. Describe the goals of sharding
        1. Determine the sharding key
        1. Articulate required application and operation changes
1. Based on these scaling patterns, a blueprint about the Data Access Layer and caching considerations
1. First iteration implementation plan
    1. Must include a plan to measure effects on both the database and the application
    1. Must include validation (testing) plan
    1. Must complete Proof of Concepts (PoC) of sharding, determine a viable path, and produce building blocks from the PoC

Work Streams and DRI

Decompose GitLab's database to improve scalability

1. Epic/Issue: https://gitlab.com/groups/gitlab-org/-/epics/6168
1. DRI: Nick Nguyen

CI/CD Data Model

1. Epic/Issue: https://gitlab.com/gitlab-org/architecture/tasks/-/issues/5

1. DRI: Grzegorz Bizon
1. Blueprint: https://gitlab.com/gitlab-org/gitlab/-/mergerequests/52203

Roles and Responsibilities

Working Group Role	Person	Title
Executive Stakeholder	Eric Johnson	Chief Technology Officer
Functional Lead	Kamil Trzci·ski	Distinguished Engineer, Ops and Enablement (Development)
Functional Lead	Jose Finotto	Staff Database Reliability Engineer (Infrastructure)
Facilitator/DRI	Nick Nguyen	Sr. Engineering Manager, Datastore
DRI - Ops Section	Sam Goldstein	Director of Engineering, Ops
DRI - Infrastructure	Steve Loyd	VP of Infrastructure
Member	Gerardo "Gerir" Lopez-Fernandez	Engineering Fellow, Infrastructure
Functional Lead	Fabian Zimmer	Group Product Manager, Enablement
Member	Stan Hu	Engineering Fellow
Member	Andreas Brandl	Staff Backend Engineer, Database
Member/CICD Data Model DRI	Grzegorz Bizon	Staff Backend Engineer (Time-decay data cibuilds)
Member	Christopher Lefelhocz	Cleaner
Member	Chun Du	Director of Engineering, Enablement
Member	Adam Hegyi	Senior Backend Engineer, Manage
Member	Tanya Pazitny	Quality Engineering Manager, Enablement & Secure
Member	Nick Westbury	Senior Software Engineer in Test, Geo
Member	Nailia Iskhakova	Senior Software Engineer in Test, Distribution
Member	Grant Young	Staff Software Engineer in Test, Memory

SECTION: company/working-groups/database-scalability/read-mostly.md

Read-Mostly Data

This document describes the read-mostly pattern introduced in the Database Scalability Working Group. We discuss the characteristics of read-mostly data and propose best practices for GitLab development to consider in this context.

Characteristics of read-mostly data

As the name already suggests, read-mostly data is about data that is much more often read than updated. Writing this data through updates, inserts, deletes is a very rare event compared to reading this data.

In addition, read-mostly data in this context is typically a small dataset. We explicitly don't deal with large datasets here, even though they often have a "write once, read often" characteristic, too.

Example: License data

Let's introduce a canonical example: License data in GitLab. A GitLab instance may have a license attached in order to use GitLab's enterprise features. This license data is held instance-wide, i.e. there typically only exist a few relevant records. This information is kept in a table licenses which is very small.

We consider this read-mostly data, because it follows above outlined characteristics:

1. Rare writes: License data very rarely sees any writes after having inserted the license.
1. Frequent reads: License data is read extremely often to check if enterprise features can be used.
1. Small size: This dataset is very small - on GitLab.com we have 5 records at < 50 kB total relation size.

Implications for the application and its database at scale

Given this dataset is small and read very often, we can expect data to nearly always reside in database caches and/or database disk caches. Thus, the concern with read-mostly data is typically not around database I/O overhead, since we typically don't read data from disk anyways. However, considering the high frequency reads, this has potential to incur overhead in terms of database CPU load and database context switches. Additionally, those high frequency queries go through the whole database stack and additional cause overhead on the database connection multiplexing components and load balancers. Also, the application spends cycles in preparing and sending queries to retrieve the data, deserialize the results and allocate new objects to represent the information gathered - all in a high frequency fashion.

In the example of license data above, the query to read license data was identified to stand out in terms of query frequency. In fact, we were seeing around 6,000 queries per second (qps) on the cluster during peak times. With the cluster size at that time, we were seeing about 1,000 qps on each replica and less than 400 qps on the primary at peak times. The difference is explained by our database load balancing for scaling reads, which favors replicas for pure read-only transactions.

The overall transaction throughput on the database primary at the time varied between 50,000 and 70,000 transactions per second (tps). In comparison to this, this query frequency only takes a small portion of the overall query frequency. However, we do expect this to still have considerable overhead in terms of context switches and hence it is worth removing this overhead, if we can.

How to recognize read-mostly data

It can be difficult to recognize read-mostly data, even though there are clear cases like in our example.

One approach to this is to look at the read/write ratio and statistics from e.g. the primary. Here, we look at the TOP20 tables by their read/write ratio over 60 minutes (taken in a peak traffic time):

```
sql
bottomk(20,
avg by (relname, fqdn) (
  (
     rate(pgstatusertablesseqtupread{env="gprd"}[1h])
     +
     rate(pgstatusertablesidxtupfetch{env="gprd"}[1h])
  ) /
  (
     rate(pgstatusertablesseqtupread{env="gprd"}[1h])
     + rate(pgstatusertablesidxtupfetch{env="gprd"}[1h])
     + rate(pgstatusertablesntupins{env="gprd"}[1h])
     + rate(pgstatusertablesntupupd{env="gprd"}[1h])
     + rate(pgstatusertablesntupdel{env="gprd"}[1h])
  )
) and on (fqdn) (pgreplicationisreplica == 0)
)
```

This yields a good impression of which tables are much more often read than written (on the database primary):

From here, we can zoom into e.g. gitlabsubscriptions and realize that index reads peak at above 10k tuples per second overall (there are no seq scans):

We very rarely write to the table (there are no seq scans):

Additionally, the table is only 400 MB in size - so this may be another candidate we may want to consider in this pattern (see 327483).

Best practices

Caching

In order to reduce the database overhead, we implement a cache for the data and thus significantly reduce the query frequency on the database side. There are different scopes for caching available:

1. RequestStore: Per-request in-memory cache (based on requeststore gem)
1. ProcessMemoryCache: Per-process in-memory cache (a ActiveSupport::Cache::MemoryStore)
1. Gitlab::Redis::Cache and Rails.cache: Full-blown cache in Redis

Continuing the above example, we had a RequestStore in place to cache license information on a per-request basis. However, that still leads to one query per request. When we started to cache license information using a process-wide in-memory cache for 1 second, query frequency dramatically dropped:

The choice of caching here highly depends on the characteristics of data in question. A very small dataset like license data that is nearly never updated is a good candidate for in-memory caching. A per-process cache is favorable here since this unties the cache refresh rate from the incoming request rate.

A caveat here is that our Redis setup is currently not using Redis secondaries and we rely on a single node for caching. That is, we need to strike a balance to avoid Redis falling over due to increased pressure. In comparison, reading data from postgres replicas can be distributed across several read-only replicas. Even though a query might be more expensive going to the database, the load is balanced across more nodes.

Read data from replica

With or without caching implemented, we also must make sure to read data from database replicas if we can. This supports our efforts to scale reads across many database replicas and removes unnecessary workload from the database primary.

GitLab's database load balancing for reads sticks to the primary after a first write or when opening an explicit transaction. In the context of read-mostly data, we strive to read this data outside of a transaction scope and before doing any writes. This is often possible given that this data is only seldomly updated (and thus we're often not concerned with reading slightly stale data, for example). However, it can be non-obvious that this query cannot be sent to a replica because of a previous write or transaction. Hence, when we encounter read-mostly data, it is a good practice to check the wider context and make sure this data can be read from a replica.

Summary & Follow-Ups

We have defined read-mostly data and described its characteristics above. We then looked at the example of license data in GitLab, the impact on GitLab.com and the effects of elevating the caching level from a request store to a per-process memory cache, which led to a significant

drop in query frequency for this data. We have described the importance of caching read-mostly data and also emphasized to check that this data is being read from a database replica whenever we can.

We acknowledged that even though we have many different caching strategies available today, there is no common framework to improve consistent and convenient use. This has been called out as a potential follow-up to define a caching framework to simplify this.

We are also aware of the limitations from a single-node Redis cache. memcached has been called out as a potential alternative for which Rails has built-in support for sharding across any number of memcached servers.

SECTION: company/working-groups/database-scalability/time-decay.md

Time-Decay data

This document describes the time-decay pattern introduced in the Database Scalability Working Group.
We discuss the characteristics of time-decay data and propose best practices for GitLab development to consider in this context.

Some datasets are subject to strong time-decay effects, in which recent data is accessed far more frequently than older data.
Another aspect of time-decay is that with time some types of data become less important, which means that we can also move old data to a bit less durable/available storage or even delete them in extreme cases.

Those effects are usually tied to product and/or application semantics and can vary in the degree that older data are accessed and how useful or required older data are to the users or the application.

Let's first consider entities with no inherent time related bias for their data.
A record for a user or a project may be equally important and frequently accessed irrelevant to when it was created. We can not predict by using a user's id or createdat how often the related record will be accessed or updated.

On the other side, a good example for datasets with extreme time-decay effects are logs and time series data, like, for example, events recording user actions.
Most of the times that type of data may have no business use after a couple of days or weeks and quickly become less important even from a data analysis perspective.
They represent a snapshot that is very quickly becoming less and less relevant to the current state of the application, until at some point they have no real value.

In the middle of the two extremes we can find datasets that have useful information that we want to keep around, but with old records seldomly being accessed after an initial (small) time period after they are created.

Characteristics of time-decay data

We are interested in datasets that show the following characteristics:

1. Size of the dataset: They are considerably large.
1. Access methods: We can filter the vast majority of queries accessing the dataset by a time related dimension or a categorial dimension with time decay effects.
1. Immutability: The time-decay status does not change.
1. Retention: whether we want to keep the old data or not and/or whether old data will be accessible by users through the application.

Size of the dataset

There can be datasets of variable sizes that show strong time-decay effects, but in the context of this blueprint we are going to focus on entities with a considerably large dataset.

Smaller datasets do not contribute significantly to the database related resource usage, nor do they inflict a considerable performance penalty to queries.

In contrast, large datasets over \~50 Million records and/or 100GB in size add a significant overhead to constantly accessing a really small subset of the data. In those cases we would want to use the time-decay effect in our advantage and reduce the actively accessed dataset.

Access methods

The second and most important characteristic of time-decay data is that most of the times we are able to implicitly or explicitly access the data using a date filter, restricting our results based on a time related dimension.

There can be many such dimensions, but we are only going to focus on the creation date as it is both the most commonly used and the one that we can control and optimize against.
It is immutable, set when the record is created, and can be tied to physically clustering the records without having to move them around.

It's important to add that even if time-decay data are not accessed that way by the application by default, there is a way to make the vast majority of the queries explicitly filter the data in such a way.
Time decay data without such a time-decay related access method are of no use from an optimization perspective as there is no way to set and follow a scaling pattern.

We are not restricting the definition to data that are always accessed using a time-decay related access method, as there may be some outlier operations, which may be necessary and we can accept them not scaling that well as long as the rest of the accesss methods can scale.
An example could be an administrator accessing all past events of a specific type while all other operations only access a maximum of a month of events, restricted to 6 months in the past.

Immutability

The third characteristic of time-decay data is that their time-decay status does not change.
Once they are considered "old", they can not switch back to "new" or relevant again.

This definition may sound trivial but we have to be able to make operations over "old" data more expensive (e.g. by archiving or moving them to less expensive storage) without having to worry about the repercussions of switching back to being relevant and having important application operations underperforming.

Consider as a counter example to a time-decay data access pattern an application view that presents issues by when they were updated. We are also interested in the most recent data from an "update" perspective, but that definition is volatile and not actionable.

Retention

Finally, a characteristic that further differentiates time-decay data in sub categories with slightly different approaches available is whether we want to keep the old data or not (e.g. retention policy) and/or whether old data will be accessible by users through the application.

(optional) Extended definition of time-decay data

As a side note, if we extend the aforementioned definitions to access patterns that restrict access to a well defined subset of the data based on a clustering attribute, we could use the time-decay scaling patterns for many other types of data.

As an example, consider data that are only accessed while they are labeled as active, like ToDos not marked as done, pipelines for not merged MRs (or a similar not time based constraint), etc.
In this case, instead of using a time dimension to define the decay, we use a categorical dimension (i.e. one that uses a finite set of values) to define the subset of interest.
As long as that subset is small compared to the overall size of the dataset, we could use the same approach.

Similarly, we may define data as old based both on a time dimension and additional status attributes, like for example ci pipelines that have failed more than 6 months ago.

Time-Decay data strategies

Partitioning

This is the acceptable best practice for addressing time-decay data from a pure database perspective.
You can find more information on table partitioning for PostgreSQL in the documentation page for table partitioning.

Partitioning by date intervals (e.g. month, year) allows us to create much smaller tables (partitions) for each date interval and only access the most recent partition(s) for any application related operation.

We have to set the partitioning key based on the date interval of interest, which may depend on two factors:

1. How far back in time do we need to access data for?

Partitioning by week is of no use if we always access data for a year back, as we would have to execute queries over 52 different partitions (tables) each time. As an example for that consider the activity feed on the profile of any GitLab user.

In contrast, if we want to just access the last 7 days of created records, partitioning by year would include too many unnecessary records in each partition, as is the case for webhooklogs.

1. How large are the partitions created?

The major purpose of partitioning is accessing tables that are as small as possible. If they get too large by themselves, queries will start underperforming and we may have to re-partition (split) them in even smaller partitions.

The perfect partitioning scheme keeps all queries over a dataset almost always over a single partition, with some cases going over two partitions and seldomly over multiple partitions being an acceptable balance. We should also target for partitions that are as small as possible, bellow 5-10M records and/or 10GB each maximum.

Partitioning can be combined with other strategies to either prune (drop) old partitions, move them to cheaper storage inside the database or move them outside of the database (archive or use of other types of storage engines).

As long as we do not want to keep old records and partitioning is used, pruning old data has a constant, for all intents and purposes zero, cost compared to deleting the data from a huge table (as described in the following sub-section).
We just need a background worker to drop old partitions whenever all the data inside that partition get out of the retention policy's period.

As an example, if we only want to keep records no more than 6 months old and we partition by month, we can safely keep the 7 latest partitions at all times (current month and 6 months in the past).
That means that we can have a worker dropping the 8th oldest partition at the start of each month.

Moving partitions to cheaper storage inside the same database is relatively simple in PostgreSQL through the use of tablespaces.
It is possible to specify a tablespace and storage parameters for each partition separately, so the approach in this case would be to:

- Create a new tablespace on a cheaper, slow disk.
- Set the storage parameters higher on that new tablespace so that the PostgreSQL optimizer knows that the disks are slower.
- Move the old partitions to the slow tablespace automatically by using background workers.

Finally, moving partitions outside of the database can be achieved through database archiving or manually exporting the partitions to a different storage engine (more details in the dedicated sub-section).

Pruning old data

If we don't want to keep old data around in any form, we can implement a pruning strategy and delete old data.

It's a simple to implement strategy that uses a pruning worker to delete past data. As an example that we'll further analyze below, we are pruning old webhooklogs older than 90 days.

The disadvantage of such a solution over large, non-partitioned tables is that we have to manually access and delete all the records that are considered as not relevant any more.
That is a very expensive operation due to multi-version concurrency control in Postgres.
It also leads to the pruning worker not being able to catchup with new records being created if that rate exceeds a threshold, as is the case of webhooklogs at the time of writing this document.

For the aforementioned reasons, our proposal is that we should base any implementation of a data retention strategy on partitioning, unless there are strong reasons not to.

Moving old data outside of the database

In most cases, we consider old data as valuable, so we do not want to prune them. If at the same time, they are not required for any database related operations (e.g. directly accessed or used in joins and other types of queries), we can move them outside of the database.

That does not mean that they are not directly accessible by users through the application; we could move data outside the database and use other storage engines or access types for them, similarly to offloading metadata but only for the case of old data.

In the simplest use case we can provide fast and direct access to recent data, while allowing users to download an archive with older data.
This is an option evaluated in the auditevents use case; depending on the country and industry, audit events may have a very long retention period, while only the past month(s) of data are actively accessed through GitLab's interface.

Additional use cases may include exporting data to a datawarehouse or other types of data stores as they may be better suited for processing that type of data. An example can be JSON logs that we sometimes store in tables: loading such data into a BigQuery or a columnar store like Redshift may be better for analyzing/querying the data.

We might consider a number of strategies for moving data outside of the database:

1. Streaming this type of data into logs and then move them to secondary storage options or load them to other types of data stores directly (as CSV/JSON data).
1. Creating an ETL process that exports the data to CSV, uploads them to object storage, drops this data from the database, and then loads the CSV into a different data store.
1. Loading the data in the background by using the API provided by the data store.

   This may be a not viable solution for large datasets; as long as bulk uploading using files is an option, it should outperform API calls.

Use cases

Web hook logs

Related epic: Partitioning: webhooklogs table

The important characteristics of webhooklogs are the following:

1. Size of the dataset: It is a really large table. At the moment we decided to partition it (2021-03-01), it had \~527M records and a total size of \~1TB

Table	Rows	Total size	Table size	Index(es) Size	TOAST Size
webhooklogs	\~527M	1.02 TiB (10.46%)	713.02 GiB (13.37%)	42.26 GiB (1.10%)	279.01 GiB (38.56%)

1. Access methods: We always request for the past 7 days of logs at max.
1. Immutability: It can be partitioned by createdat, an attribute that does not change.
1. Retention: There is a 90 days retention policy set for it.

Additionally, we were at the time trying to prune the data by using a background worker (PruneWebHookLogsWorker), which could not keep up with the rate of inserts.

As a result, on March 2021 there were still not deleted records since July 2020 and the table was increasing in size by more than 2 million records per day instead of staying at a more or less stable size.

Finally, the rate of inserts has grown to more than 170GB of data per month by March 2021 and keeps on growing, so the only viable solution to pruning old data was through partitioning.

Our approach was to partition the table per month as it aligned with the 90 days retention policy.

The process required follows:

1. Decide on a partitioning key

   Using the createdat column is straightforward in this case: it is a natural partitioning key when a retention policy exists and there were no conflicting access patterns.

1. Once we decide on the partitioning key, we can start with creating the partitions and backfill them (copy data from the existing table).

   The reason for that is that we can't just partition an existing table, we have to create a new partitioned table.

   So, we have to create the partitioned table and all the related partitions, start copying everything over and also add sync triggers so that any new data or updates/deletes to existing data can be mirrored to the new partitioned table.

   MR with all the necessary details on how to start partitioning a table

It required a 15 days and 7.6 hours to complete that process.

1. Next step, one milestone after the initial partitioning starts is to clean up after the background migration used to backfill and finish executing any remaining jobs, retry failed jobs, etc.

MR with all the necessary details

1. Then we can add any remaining foreign keys and secondary indexes to the partitioned table.

The purpose of this operation is to bring its schema on par with the original non partitioned table before we can swap them in the next milestone.

We are not adding them at the beginning as they are adding overhead to each insert and they would slow down the initial backfilling of the table (in this case for more than half a billion records, which can add up significantly). So we create a lightweight, vanilla version of the table, copy all the data and then add any remaining indexes and foreign keys.

MRs with all the necessary details: MRs adding indexes(MR-1, MR-2), MR adding Foreign Keys

1. Swap the base table with partitioned copy

This is the point when the partitioned table starts actively being used by the application.

Dropping the original table is a destructive operation and we want to make sure that we had no issues during the process, so we keep the old non-partitioned table. We also switch the sync trigger the other way around so that the non-partitioned table is still up to date with any operations happening on the partitioned table. That allows us to swap back the tables if it is necessary.

MR with all the necessary details

1. Last step, one milestone after the swap - Drop the non-partitioned table

Issue with all the necessary details

1. After the non-partitioned table is dropped, we can add a worker to implement the pruning strategy by dropping past partitions.

In this case, the worker will make sure that only 4 partitions are always active (as the retention policy is 90 days) and drop any partitions older than four months. We have to keep 4 months of partitions while the current month is still active, as going 90 days back takes you to the fourth oldest partition.

Audit Events

Related epic: Partitioning: Design and implement partitioning strategy for Audit Events

The auditevents table shares a lot of characteristics with the webhooklogs table discussed in the previous sub-section, so we are going to focus on the points they differ.

There was a need for a solution (1, 2) and a consensus that partitioning could solve most of the performance issues (1, 2).

In contrast to most other large tables, it has no major conflicting access patterns: we could switch the access patterns to align with partitioning by month.
This is not the case for example for other tables, which even though could justify a partitioning approach (e.g. by namespace), they have many conflicting access patterns.

In addition, auditevents is a write heavy table with very few reads (queries) over it and has a very simple schema, not connected with the rest of the database (no incoming or outgoing FK constraints) and with only two indexes defined over it.

The later was important at the time as not having Foreign Key constraints meant that we could partition it while we were still in PostgreSQL 11. This is not a concern any more now that we have moved to PostgreSQL 12 as a required default, as can be seen for the webhooklogs use case above.

The migrations and steps required for partitioning the auditevents are similar to the ones described in the previous sub-section for webhooklogs. There is no retention strategy defined for auditevents at the moment, so there is no pruning strategy implemented over it, but we may implement an archiving solution in the future.

What's interesting on the case of auditevents is the discussion on the necessary steps that we had to follow to implement the UI/UX Changes needed to encourage optimal querying of the partitioned. It can be used as a starting point on the changes required on the application level to align all access patterns with a specific time-decay related access method.

CI tables

WIP: Requirements and analysis of the CI tables use case - Still a work in progress, so we'll have to add more details after the analysis moves forward.

SECTION: company/working-groups/isolation/_index.md

Attributes

Property	Value
Date Created	October 7, 2019
Date Closed	February 28, 2020
Slack	wgisolation (only accessible from within the company)
Google Doc	Isolation Working Group Agenda (only accessible from within the company)

Business Goal

To develop a plan that limits disruption to customers when there are "noisy neighbors" or when other unexpected events occur.

Exit Criteria

1. Determine what is in-scope for the working group and select specific lines of enquiry
1. For each line of enquiry
   - determine the appropriate goal
   - define backlog of issues
   - provide estimated timeline
   - schedule issues for work with groups
1. Create guidance on how to address isolation issues

Group Closure Information: All lines of inquiry were incorporated into the Availability and Performance Weekly Meeting.

Specific Lines of Enquiry

- Application Level Redis Sharding (Grzegorz Bizon)
- File Storage Isolation (Marin Jankovski)
- Database Partitioning (Craig Gomes)

Decoupled Service (Craig Gomes) was removed from this working group
as this will be a future effort for the Memory group.

Roles and Responsibilities

Working Group Role	Person	Title
Executive Stakeholder	Christopher Lefelhocz	Senior Director of Development
Facilitator	Rachel Nienaber	Engineering Manager, Geo
DRI for Application Level Redis Sharding	Grzegorz Bizon	Staff Backend Engineer, Configure:System
DRI for File Storage Isolation	Andrew Newdigate	Distinguished Engineer, Infrastructure
DRI for Database Partitioning	Craig Gomes	Engineering Manager, Memory
Functional Lead	Ramya Authappan	Quality Engineering Manager, Dev
Functional Lead	Fabian Zimmer	Senior Product Manager, Geo
Functional Lead	Gerardo "Gerir" Lopez-Fernandez	Engineering Fellow, Infrastructure
Functional Lead	Stan Hu	Engineering Fellow, Development
Member	George Burdell	Campus Alumni Representative
Member	Chun Du	Director of Engineering, Enablement

| Member | Wayne Haber | Director of Engineering, Protect |
| Member | Nicholas Klick | Engineering Manager, Configure:System |

SECTION: company/working-groups/isolation/fault_tolerance.html.md

Fault-Tolerance

GitLab has to be a highly-available, mission critical system. To achieve this, we must design and deploy the system in such a way that a number of principles are met:

1. Eliminate single points of failures (SPOF): A failure of a single node should not cause downtime.
1. Isolate failures: If a failure happens, it should be isolated as much as possible to a particular project, user, etc. The blast radius should be minimized.
1. Rollback: Errors will invariably happen in the software development. If a bug happens, we must be able to revert quickly without exposing the problem to a large number of users.

Example Improvements to GitLab

Below is a list of examples of concrete items that will help improve GitLab fault-tolerance:

SPOF

1. Eliminate use of NFS
   1. https://gitlab.com/gitlab-com/gl-infra/scalability/issues/62
   1. https://gitlab.com/gitlab-org/gitlab/issues/32203
1. Use multiple Redis cache instances in Rails.cache

Isolation

Isolation epic

1. Allow GitLab to function if a single Gitaly node is down
   1. https://gitlab.com/gitlab-org/gitlab/issues/34722
   1. https://gitlab.com/gitlab-org/gitlab/issues/39509
1. TODO

Microservices does not necessarily provide fault isolation

Note that the above list does not mention microservices as a cure-all. A microservice architecture can help provide fault isolation, but it does not inherently do this. For example, let's suppose we introduce

UserAPI microservice that creates an API for all services to retrieve users in the system. Now our architecture may look like:

mermaid
graph TD
  Rails --> UserAPI
  Sidekiq --> UserAPI
  Pages --> UserAPI

The UserAPI microservice could still be a single point of failure here; if that goes down, all the other services in the system (e.g. Rails, Sidekiq, etc.) also stop working. We've introduced a new service that can be owned by a single team, but in doing so we haven't necessarily improved isolation. Can the system function without this service? Probably not, although there may be other advantageous to doing this (e.g. make it possible to shard user data in multiple servers, performance, etc.). We still have to think about how to avoid a SPOF.

In addition, GitLab also is unique in that every microservice that we create has to be shipped to customers, so there is overhead in managing configuration and redundancy of these services as well.

That being said, microservices may be worth it if we can clearly define the engineering benefit towards maintainability, scalability, and reliability. For example, we've considered introducing a GitLab CI service daemon that can better handle CI queues.

SECTION: company/working-groups/real-time/_index.md

Attributes

Property	Value
Date Created	March 12, 2020
Date Closed	November 1, 2021
Slack	wgreal-time (only accessible from within the company)
Google Doc	Real-Time Working Group Agenda (only accessible from within the company)
Epic & Design Doc	Use ActionCable for real-time features
Feature Issue	View real-time updates of assignee in issue / merge request sidebar
Associated OKRs	Plan: Support incremental ACV

Business Goal

To ship one real-time feature to self-managed customers.

Exit Criteria - Phase 1

(· Done, ·· In-progress)

One Real-Time Feature, Usable by Single Instance/Small Cluster, Self-Hosting Customers => 100%

Issue

- Supports starting Action Cable in embedded mode ·
- Omnibus includes ability to start embedded Action Cable with config/cable.yml ·
- GDK supports configuration of Action Cable ·
- Workhorse Proxies Action Cable requests ·
- Backend work is complete to upgrade websocket connections and push signal when assignees are updated on an issue ·
- Frontend work is complete to respond to WebSockets with an update to the sidebar ·
- Action Cable settings are exported to Prometheus ·
- Track ActionCable settings in Usage Ping ·
- Documentation enabled to advise customers on using the feature in small deployments ·
- Feature usable conditionally when Action Cable is enabled ·

Exit Criteria - Phase 2

(· Done, ·· In-progress)

One Real-Time Feature, Usable on larger deployments => 100%

Issue

- Omnibus includes ability to start standalone Action Cable Puma Server with config/cable.yml ·
- GDK allows configuration of standalone Action Cable and starts Puma server ·
- Workhorse Proxies Action Cable requests ·
- Establish reference architectures for supporting WebSocket connections at scale ·
- Helm charts allow configuration of embedded Action Cable in webservice service ·
- QA work complete to test initial feature ·
- Feature flag defaulted to on, with suitable fallback ·

Working, Real-Time Feature Available on .com => 100%

Epic

- Containerization of Real-Time Feature including Action Cable and Puma ·
- Update of Helm charts allowing use of multiple Redis instances ·
- WebSocket requests supported in Kubernetes ·
- Deployment to a non-production environment for manual, full-stack testing ·
- Action Cable requests served by Kubernetes pods on Staging ·
- Action Cable requests served by Kubernetes pods on Canary ·
- Readiness-review complete to ensure observabilty & contain risk ·
- Action Cable requests served by Kubernetes pods on Production ·
- Default to embedded Action Cable enabled ·

Design Document

Technical decisions and rationale are captured in this design document.

Roles and Responsibilities

Working Group Role	Person	Title
Executive Sponsor	Christopher Lefelhocz	VP of Development
Facilitator	John Hope	Engineering Manager, Plan
Functional Lead	Heinrich Lee Yu	Senior Backend Engineer, Plan
Functional Lead	Gabe Weaver	Senior Product Manager, Plan
Functional Lead	Sean McGivern	Staff Backend Engineer, Scalability
Member	Scott Stern	Frontend Engineer, Plan
Member	Ben Kochie	Site Reliability Engineer
Member	Natalia Tepluhina	Staff Frontend Engineer, Plan
Member	Matthias Käppler	Senior Engineer, Memory
Member	Jake Lear	Engineering manager, Plan

Meetings

Meetings are recorded and publicly available on
YouTube in the Real-Time Working Group playlist.

<iframe width="560" height="315"
src="https://www.youtube.com/embed/videoseries?list=PL05JrBw4t0KoMOcLID1fKWWR4H2n2hQ"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>

SECTION: company/working-groups/real-time/design_document.md

Introduction

This document captures the design decisions behind the current implementation of real-time features in GitLab. It is adapted from the description and discussion within issues in this epic.

The Problem

We want to implement real-time issue boards but the current way of doing real-time (polling with ETag caching) would not work well with it. We'd have to track a lot of things in an issue board (lists changing, issues within lists changing, etc..) and polling for each of those isn't feasible due the number of requests that would be required and the resultant load on server nodes and the database. It is possible to use just one polling endpoint, but it just makes the code harder to understand.

Polling is also not very real-time unless polling intervals can be dropped substantially.

Even pages that successfully use multiple polling requests; such as the MR page for title and description, notes, widgets, and so on, would benefit from faster updates.

<!--
We decided to explore Pub/Sub messaging systems because it would make these simpler. On a page, we could subscribe to multiple channels / events and they would go through one connection.

This could also potentially take some load off Redis and our web servers when we eliminate those polling requests. We'd have extra load going to the websocket server and Redis for Pub/Sub but I think that's more efficient so we'd still see a gain overall.
-->

Objectives

The objective is to implement a real-time solution that satisfies the following criteria:

- Enables low-latency, bidirectional communication between client and server;
- Deployable on self-managed instances of all sizes and GitLab.com at scale;
- Optional and degrades gracefully;
- Safely isolated from critical infrastructure;
- Reuses existing code and established technology (i.e. is a boring solution) where possible; and
- Facilitates an iterative approach, starting with a small, low-risk feature.

This objective is an iterative step in the long-term plan to implement real-time collaboration.

Solution Proposed

We will roll out the use of WebSockets by starting with a small, relatively low-risk feature. When we've identified and solved the problems of maintaining WebSocket connections at scale and captured the lessons of designing a feature for persistent connections, we'll produce documentation that will allow other developers to work on real-time features.

The initial feature is viewing assignees on issues in real-time and the chosen technology is Action Cable.

How it works

For the simplest deployments, enabling Action Cable enables the first feature by default.

The feature can also be toggled using two feature flags:

realtimeissuesidebar	Attempts to establish a WebSocket connection when viewing an issue and responds to update signals
broadcastissueupdates	Broadcasts a signal when an issue is updated

Sometimes, the nodes serving Web requests aren't the same ones serving WebSocket connections (see "How to implement it on premise") so don't have Action Cable enabled. The feature flags can be used to enable the feature explicitly.

This diagram shows the current steps involved in establishing an open WebSocket connection for

bidirectional communication. This is subject to change as work progresses.

mermaid
    sequenceDiagram
      participant Client
      participant Workhorse
      participant Rails/AC

      Client->>Workhorse: HTTP GET Upgrade: websocket
      Workhorse->>Rails/AC: proxy
      Rails/AC->>Rails/AC: open connection
      Rails/AC-->>Workhorse: HTTP 101 Switching Protocols
      Workhorse-->>Client: 200 OK
      Rails/AC->>Client: websocket traffic
      Client->>Rails/AC: websocket traffic

1. The client sends a connection upgrade request to /-/cable;
1. Workhorse proxies this to the correct backend (set using the cableBackend option, defaulting to authBackend);
1. The backend responds with 101 Switching Protocols and upgrades the request;
1. The client subscribes to the channel(s) it's interested in (IssuesChannel, specifying projectpath and iid);
1. The server confirms subscription and publishes a signal when an update is made to the issue; and
1. The client responds by requesting up-to-date state via GraphQL[·].

·: This step is especially subject to change as we consider using GraphQL Subscriptions instead.

How it solves the problem

Prototype model / Testing plan

The feature is currently available for internal team-members to demo on the dev.gitlab.org instance. This is a single-instance deployment of CE.

Performance testing Action Cable with Puma determined no impact on resource usage but only tested while idle. In the absence of simulated workloads, the recommendation was to roll the feature out gradually.

An end-to-end test for real-time assignees was added in this MR.

How to implement it on premise

Instance administrators have a number of options for using Action Cable. The simplest way is to serve WebSocket connections from existing nodes, and this is enabled by default since 14.5.

While extensive testing and experience supporting WebSockets at scale on large deployments

internally indicate this is probably not necessary, administrators of larger deployments may wish to proxy WebSocket connections to a separate set of nodes to protect their main Web nodes from saturation. This can be done by splitting WebSocket traffic at the load balancer or ingress stage, typically based on path. This is the method used for gitlab.com.

Only embedded mode is supported for Action Cable. Even when splitting traffic, all nodes are running full GitLab Web processes with Action Cable enabled. See the decision to support only embedded mode here.

It's important to note that Action Cable channels (similar to controllers) can do anything that can be done in the web context; such as using models or reading from the cache, so it is important that these processes are treated like existing web processes. They should have the same configuration and should be able to connect to the DB, Redis cache, shared state, sidekiq, etc. Although we probably would just be doing permission checks in the initial implementation, it could be a source of weird bugs in the future if these dependencies aren't setup properly.

How to implement it on .com

1. Infrastructure supporting WebSocket connections will run in Kubernetes;
1. WebSocket traffic is split by path to a separate, independently scalable deployment; and
1. Pods servicing WebSocket connections run ordinary webservice processes with Action Cable enabled.

Initially, nodes serving Web requests did not have Action Cable enabled on gitlab.com. The feature had to be controlled using the feature flags :realtimeissuesidebar and :broadcastissueupdates and these were used to roll-out in a controlled way. This setup was simplified so that nodes have Action Cable enabled, regardless of whether they serve WebSocket requests or Web requests. This allows the nodes serving Web requests to know that Action Cable is available without relying on a feature flag.

Monitoring

- Prometheus metrics on thread pool size and active connections are sampled by the ActionCableSampler.
- Additional metrics implemented in gitlab-org/gitlab296845 will allow instance administrators, including on GitLab.com, to capture useful metrics suitable for building a standard service dashboard; such as message counts and sizes.
- GitLab.com has (internal) dashboards for WebSocket SLIs, Kubernetes containers and the overall deployment.

Possible Costs

Based on the rollout of the first real-time feature, the current esimated cost per connection at peak is $0.02 (USD) per month.

This figure is derived by dividing total cost of WebSocket nodes by the number of concurrent connections at peak, visible on this chart (internal). It does not take into account load on downstream services, such as the primary database or Redis. It is most likely an over-estimation and expected to decrease as connections are added, as the current nodes can support more connections.

Alternatives Considered

Action Cable was the first choice because it is included with Rails. Scalability is a known concern but if it becomes a problem Anycable implements the same API. We could switch to that in the future with minimal to no changes in the application code.

1. Long-polling / Server-sent Events (SSE)

Both long-polling and SSE have the problem detailed above with having to poll / request multiple endpoints. Also, even if we do this, we'd have to implement some custom backend logic similar to our current ETag caching that checks Redis or something similar. It's not worth it when ActionCable provides the full stack.

The messagebus gem implements multiple subscriptions in one polling endpoint. But since we're planning to do real-time collaboration which would need lower latencies and bi-directional communication, it's better to just go with websockets directly.

1. Go / Erlang / Elixir websocket servers

It is known that these languages are better than Ruby at concurrency but without booting our Rails app / Ruby libraries, we can't reuse the code we already have; for example, permissions checks. These are complex and very easy to get wrong so we definitely don't want to re-implement this. We could do a separate API call to our Rails backend but more on that below.

1. Anycable

Anycable has websocket servers in Go / Erlang and it solves the problem of not having Rails context by using gRPC. The downside is that we'd have to spin up another gRPC server which boots our Rails app. This complicates our infrastructure setup and would take longer to setup everything that's needed. This option was discussed in this issue.

Since it is easy to switch to this later on if needed, we decided to defer this and start with Action Cable.

1. Other Ruby websocket servers (Faye)

Gitter uses this and we looked into this briefly but we didn't really have a strong reason to choose this over Action Cable.

Documentation

- Omnibus settings
- Real-time issue sidebar user documentation
- Developer Documentation
- Performance testing
- Readiness review for GitLab.com

SECTION: company/working-groups/secure-govern-database-decomposition/_index.md

Attributes

Property	Value
Date Created	1 May 2024
Start Date	13 May 2024
End Date	
Slack	wgsec-database-decomposition (only accessible from within the company)
Google Doc	Working Group Agenda (only accessible from within the company)
Issue Board	Epic Dashboard list
Meeting Cadence	Weekly on Mondays. Recorded. EMEA and APAC options.

Exit Criteria

The charter of this working group is to:

- Successfully decompose the Sec datasets to a separate gitlabsec database in order to reduce pressure on the primary GitLab.com DB and assist in future scalability and stability concerns.
- Consider the timing, scope, and impact of the decomposition related to prioritization and implementation of additional efforts to support GitLab.com db performance and optimization for related tables - OKR (GitLab internal)
- Evaluate the impact of the decomposition on Self-Managed instances regarding feature parity, performance/hardware requirement, improvements for different size of DBs, and admin's effort to support.

Support for decomposition in Self-managed or Dedicated is not in scope for this working group.

Objectives

Key results we'd like to achieve within the scope of the working group to ensure the outcome has the most desireable outcome.

Objective	Notes	Achieved On
To the best of our ability, ensure implementation does not result in increased costs or burden for self-managed users, similar to the CI decomposition outcome.		
Modifications have been signed off on by Reference Architecture and appropriately documented.	Raise an issue in the Reference Architecture tracker to gather needed advice on an ad hoc basis.	
Minimise disruption to GitLab.com in the process of decomposition.	This may be unavoidable if we opt to use Phsyical Replication, which will require all traffic to the database to be ceased prior to cutover.	

Glossary

Preferred Term	What Do We Mean	Terms Not To Use	Examples
Cluster	A database cluster is a collection of interconnected database instances that replicate data.		The PostgreSQL cluster of GitLab.com (managed by Patroni) that hosts the main logical database and consists of the primary database instance along with its read-only

replicas. |
Decomposition	Feature-owned database tables are on many logical databases on multiple database instances. In terms of GitLab.com, our desired decomposition outcome includes the separation of these instance to different database servers as well. The application manages various operations (ID generation, rebalancing etc.)	Y-Axis, Vertical Sharding	All Sec tables in separate logical database from Core tables. Design illustration
Instance	A database instance is comprised of related processes running in the database server. Each instance runs its own set of database processes.	Physical Database	
Logical database	A logical database groups database objects logically, like schemas and tables. It is available within a database instance and independent of other logical databases.	Database	GitLab's rails database.
Node	Equivalent to a Database Server in the context of this working group.	Physical Database	
Replication	Replication of data with no bias.	X-Axis, Cloning	What we do with our database clusters to enable splitting read traffic apart from write traffic.
WAL (Write Ahead Log)	Write ahead logs are the mechanism by which Postgres records inserted data. WAL records are then processed to modify the stored dataset in a separate process. These logs can be replicated.		
Logical Replication	Replication of data using the built-in Postgres replication processes to transfer WAL via a PUB-SUB model		
Physical Replication	Replication of data by copying the actual files on the written disk to a new Phsyical Database.		
Application Replication	Replication of data to a separate database by the configuration of replication routines in GitLab itself.		
DB Schema	A SQL database schema is a namespace that contains named database objects such as tables, v